

CIC0198 - TÉCNICAS DE PROGRAMAÇÃO 2

Arquiteturas atuais

Daniel Porto – daniel.porto@unb.br
Departamento de Ciência da Computação – CIC
Universidade de Brasília – UnB
Material adaptado do Prof. Marco Tulio Valente



UnB | IE | CIC

Arquiteturas atuais

Domain-Driven Design (DDD)

Arquitetura Hexagonal

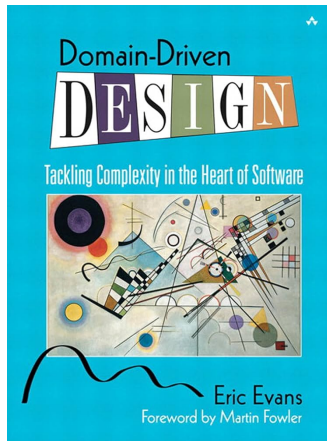
Arquitetura Limpa (Clean Architecture)

Injeção de Dependência

Padrão Composite

Arquitetura Serverless

DOMAIN-DRIVEN DESIGN (DDD)



2003 (“the blue book”)

PROJETO DE SOFTWARE

Duas preocupações: *domínio e tecnologia*

- ▶ **Domínio:** problema que sistema resolve
- ▶ **Tecnologia:** interface com usuário, persistência, etc

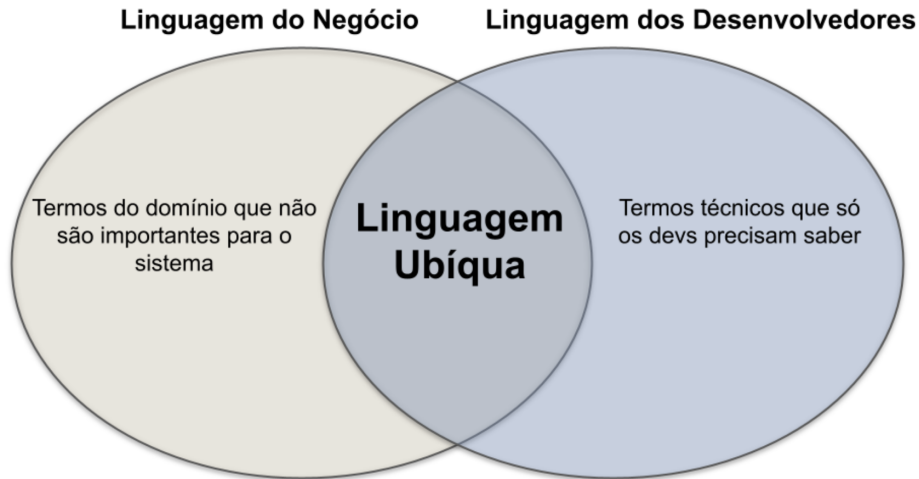
Ambas são importantes, mas DDD defende:

- ▶ Separação entre essas duas preocupações
- ▶ Maior atenção ao domínio, principalmente se complexo

CONCEITOS IMPORTANTES EM DDD

- ▶ Linguagem Ubíqua
- ▶ Objetos de Domínio
- ▶ Contextos Delimitados

LINGUAGEM UBÍQUA



SISTEMA DE BIBLIOTECAS

Linguagem ubíqua:

- ▶ *Livro, Exemplar, ISBN, Bibliotecária, Usuário, Acervo, Reserva, Empréstimo, Multa, Catálogo, etc*

Linguagem apenas dos devs:

- ▶ *Proxy, observadores, cache, camadas, rotas, etc*

Linguagem apenas das bibliotecárias:

- ▶ *ISBN alternativos*

LINGUAGEM UBÍQUA (EM TERMOS PRÁTICOS)

- 1 Glossário de termos (nomes + definição)
- 2 Relacionamentos entre termos (por exemplo, via diagramas de classe simplificados)

EXEMPLO 2: RELACIONAMENTOS ENTRE TERMOS

- ▶ Um Livro pode ter um ou mais Exemplares
- ▶ Uma Reserva pode ser feita para no máximo três Livros
- ▶ Existem 3 tipos de Usuário: Aluno, Professor e Externo
- ▶ O Acervo é formado por um conjunto de Livros

EXEMPLO: TERMOS QUE PODEM APARECER DUPLICADOS NO CÓDIGO DE ALGUNS SISTEMAS

- ▶ Universidade: Aluno e Estudante
- ▶ iFood: Restaurante e Estabelecimento
- ▶ Amazon: Cliente e Usuário

EXEMPLO: TERMOS QUE PODEM APARECER DUPLICADOS NO CÓDIGO DE ALGUNS SISTEMAS

- ▶ Universidade: Aluno e Estudante
- ▶ iFood: Restaurante e Estabelecimento
- ▶ Amazon: Cliente e Usuário

Significam a mesma coisa e, portanto, deveriam ser padronizados

OUTROS EXEMPLOS

- ▶ Customer, visitor, buyer, purchaser, and member
- ▶ Guests, customers, patrons, and diners
- ▶ Employee, associate, member, and staff
- ▶ Order, request, cart, and ticket

Fonte: Software Requirements, 3rd Edition (Karl Wieggers, Joy Beatty)

OBJETIVOS E USOS DA LINGUAGEM UBÍQUA

- ▶ Comunicação com stakeholders
- ▶ Nomear elementos do código fonte (métodos, variáveis, classes, pacotes, diretórios, exceções, etc)

OBJETOS DE DOMÍNIO (“DDD BUILDING BLOCKS”)

Domínio deve ser formado pelos seguintes objetos:

- ▶ Entidades
- ▶ Objetos de Valor
- ▶ Serviços
- ▶ Agregados
- ▶ Repositórios

ENTIDADES

Objeto que possui uma identidade única, que o distingue dos demais objetos da mesma classe.

Exemplos:

- ▶ Usuário (de uma biblioteca)
- ▶ Aluno (de uma universidade)
- ▶ Pergunta (em um fórum de perguntas e respostas)

2. OBJETOS DE VALOR (“VALUE OBJECTS”)

Não possuem um identificador único

São caracterizados pelo seu estado (valores de seus atributos).

Exemplos:

- ▶ Endereço, Moeda, Data, telefone, Email, Hora, Cor, etc.

POR QUE DISTINGUIR ENTIDADES E OBJETOS DE VALOR?

Entidades:

- ▶ Objetos mais complexos, costumam ser tabelas do BD
- ▶ Regras específicas para criação e remoção

Objetos de Valor:

- ▶ Mais simples
- ▶ Devem ser imutáveis
- ▶ Métodos de validação (data é válida?)

EXEMPLO: EMAIL

Em memória, objetos do tipo Email devem ser imutáveis

Como alterar um email? Criar novo objeto, com novo valor

No BD, emails são campos de uma tabela

Exemplo: tabela Pessoa possui um campo email

- ▶ Evidentemente, esse campo é mutável

3. SERVIÇOS

Objetos que agrupam operações importantes do domínio
Normalmente, são stateless e singletons

EXEMPLO DE UMA CLASSE/OBJETO DE SERVIÇO

```
1 class ServicoDeEmprestimo {  
2     ...  
3     void emprestarLivro(Usuario, Livro) {...}  
4     void devolverLivro(Usuario, Livro) {...}  
5     ...  
6 }
```

4. AGREGADOS

Um agregado é um grupo de entidades e objetos de valor que forma uma unidade semântica

Do ponto de vista de projeto, os objetos de um agregado devem ser analisados juntos

AGREGADOS

Um agregado possui um **objeto raiz** (uma entidade)

- ▶ Acesso externo é sempre via raiz
- ▶ Raiz: referencia objetos internos

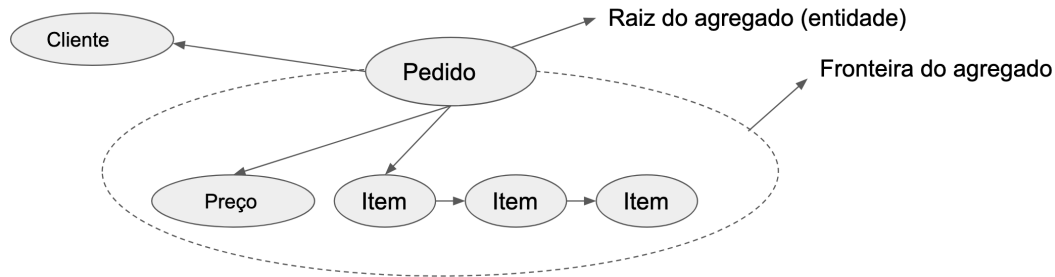
São persistidos em conjunto em bancos de dados

Deleção: deleção da raiz e de todos os objetos internos

EXEMPLO DE AGREGADO

Pedido (classe; entidade), com dois atributos:

- ▶ **Preço** (classe; objeto de valor)
- ▶ **Lista de ItemPedido** (classe; objeto interno)



5. REPOSITÓRIO

- ▶ Objeto que recupera outros objetos do domínio de um BD
- ▶ **Objetivo:** prover uma abstração que blindar os desenvolvedores de preocupações relacionadas com BDs
- ▶ Usados para recuperar, deletar, atualizar, etc entidades ou agregados

EXEMPLO DE REPOSITÓRIO

```
1 class RepositorioDeEmprestimos {
2     List<Emprestimo> findEmprestimosDeUsuario(Usuario u) {...}
3     List<Emprestimo> findEmprestimosPorData(Data inicio, Data fim) {...}
4     List<Emprestimo> findEmprestimosVencidos() {...}
5     ...
6     void salvar(Emprestimo e) {...}
7     void atualizar(Emprestimo e) {...}
8     void remover(Emprestimo e) {...}
9 }
```

CONTEXTOS DELIMITADOS (“BOUNDED CONTEXTS”)

Grandes organizações possuem vários modelos de domínio

Exemplo: financeiro, pessoas, logística, clientes, etc

Contextos delimitados: subdomínios de um domínio maior

Cada contexto delimitado possui:

- ▶ Uma linguagem de domínio própria
- ▶ Um modelo de domínio próprio

EXERCÍCIO 1

Suponha que você trabalha em uma empresa que possui um aplicativo para entrega de comida. Você ficou responsável pelo projeto da camada de domínio do sistema e decidiu usar DDD. Descreva então:

- a) cinco termos da linguagem ubíqua do sistema
- b) três entidades
- c) três objetos de valor
- d) um agregado (incluindo objeto raiz e objetos internos)
- e) dois métodos de um serviço
- f) dois métodos de um repositório

EXERCÍCIO 2

Seja o seguinte método do sistema de bibliotecas (que usa DDD):

```
1 void emprestarLivros(int userId, int livroId) {  
2     Usuario usuario = "recuperar dados de userId"  
3     Livro livro = "recuperar dados de livroId"  
4     usuario_ok = "chechar se usuário em dia com a biblioteca"  
5     livro_disponivel = "chechar se livro tem exemplares disponíveis"  
6     if (usuario_ok && livro_disponivel)  
7         "criar empréstimo de livro para usuario"  
8 }
```

- a) emprestarLivro() pertenceria a qual tipo de classe?
- b) Como você classificaria as classes Usuario e Livro?
- c) As operações "recuperar dados de usuário", "recuperar dados de livro" e "criar empréstimo" pertenceriam a qual tipo de classe?

EXERCÍCIO 2 (CONTINUAÇÃO)

- d) Suponha que Emprestimo (uma classe) tenha um conjunto de ItemEmprestimo (outra classe). O conjunto dessas duas classes forma que tipo de estrutura em DDD?

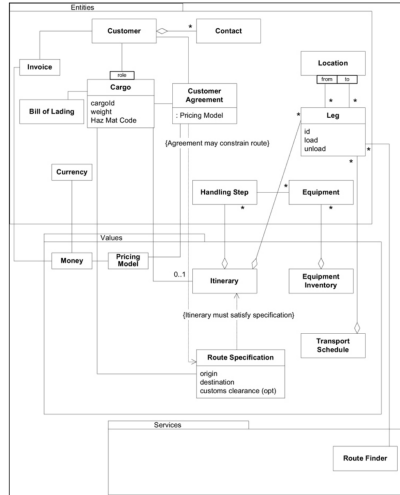
EXERCÍCIO 3

Depois de aprender DDD, um desenvolvedor resolveu estruturar seu sistema como mostrado a seguir. Ele criou um pacote (ou um módulo ou diretório) para agrupar os arquivos que implementam os tipos de objetos de domínio preconizados por DDD. Essa decisão é consistente com os princípios de DDD? Justifique.

Raiz

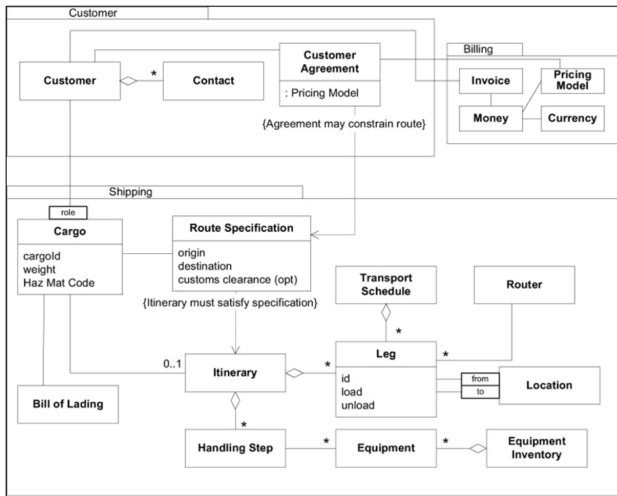
```
|__ Entidades
|   | arquivos que implementam entidades
|
|__ Agregados
|   | arquivos que implementam agregados
|
|__ ObjetosValor
|   | arquivos que implementam objetos de valor
|
|__ Servicos
|   | arquivos que implementam serviços
|
|__ Repositorios
|   | arquivos que implementam repositórios
```

SOLUÇÃO MENOS RECOMENDADA



Fonte: DDD Book, chapter 7, fig. 7.7

SOLUÇÃO MAIS RECOMENDADA



Fonte: DDD Book, chapter 7, fig. 7.8

Arquiteturas atuais

Domain-Driven Design (DDD)

Arquitetura Hexagonal

Arquitetura Limpa (Clean Architecture)

Injeção de Dependência

Padrão Composite

Arquitetura Serverless

ARQUITETURA HEXAGONAL

Objetivo de uma Arquitetura Hexagonal:

Estabelecer uma separação clara entre:

- ▶ **Domínio** (classes do negócio)
- ▶ **Restante do sistema**

Restante do sistema:

- ▶ Interface com o usuário (Web, mobile, etc)
- ▶ Persistência e BDs
- ▶ Integrações com outros sistemas
- ▶ etc

VANTAGENS (DE SEPARAR DOMÍNIO DO RESTO)

- 1 Permite que desenvolvedores se concentrem no domínio:
 - ▶ Representa o propósito do sistema
 - ▶ Responsável pela geração de valor
- 2 Testabilidade (mais fácil testar domínio limpo de tecnologia)
- 3 Fica mais fácil trocar de bibliotecas, frameworks, BDs, etc.

Observação: (1) e (2) costumam ser bem mais importantes do que (3).

ARQUITETURA HEXAGONAL

Definida por Alistair Cockburn, em meados dos anos 90

Ideia central: usar adaptadores para mediar a comunicação entre domínio e o resto do sistema

Adaptadores: mesma ideia do padrão de projeto

- Frontend (Web, mobile, etc)
- Bancos de dados
- Sistemas externos (gateways pagto, etc)
- Bibliotecas de terceiros (logging, cache, pub/sub, etc)



Adaptadores

Domínio “limpo” de tecnologia

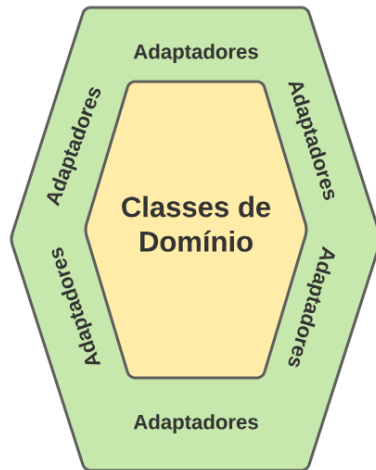
DOMÍNIO LIMPO DE TECNOLOGIA

Camada de domínio não conhece:

- ▶ Banco de dados usados pelo sistema
- ▶ Frontend usados pelo sistema
- ▶ Gateways de pagamentos usados pelo sistema
- ▶ Serviços externos com os quais o sistema interage
- ▶ etc

ARQUITETURA HEXAGONAL

Nome deriva do diagrama usado para ilustrar a arquitetura, formado por dois hexágonos concêntricos



FACES DO HEXÁGONO

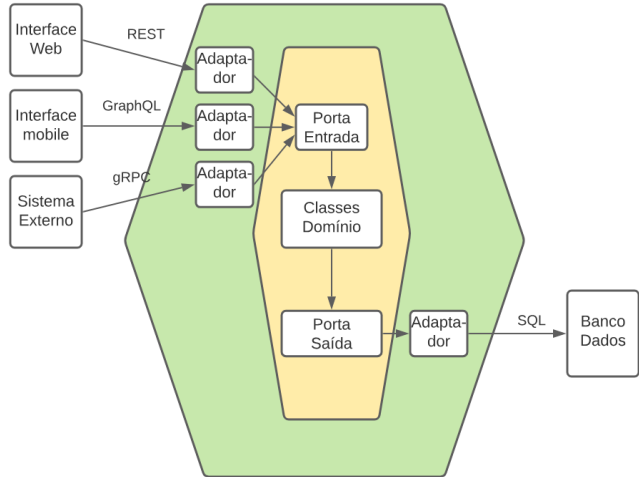
“Cada face do hexágono representa um motivo pelo qual o sistema deve se comunicar com o mundo exterior. É por isso que são hexágonos e não círculos concêntricos.”

Motivos para comunicação com o mundo exterior:

- ▶ Interagir com usuários (interface com usuários)
- ▶ Persistir dados em um BD
- ▶ Enviar informações para outros sistemas
- ▶ etc

EXEMPLO: SISTEMA DE BIBLIOTECAS

Arquitetura Hexagonal é também chamada de arquitetura baseada em **portas e adaptadores**



EXERCÍCIO 1

Em uma Arquitetura Hexagonal, um adaptador é uma implementação do padrão de projeto de mesmo nome. E as portas? Elas podem ser vistas como sendo uma implementação – pelo menos aproximada – de qual padrão de projeto?

EXERCÍCIO 1

Em uma Arquitetura Hexagonal, um adaptador é uma implementação do padrão de projeto de mesmo nome. E as portas? Elas podem ser vistas como sendo uma implementação – pelo menos aproximada – de qual padrão de projeto?

Portas são semelhantes a **fachadas**, pois elas simplificam e uniformizam a comunicação com um sistema externo, normalmente mais complexo.

EXERCÍCIO 2

A definição do termo hexagonal é arbitrária, pois, dependendo da aplicação, ela poderia ser chamada de quadrangular, pentagonal, heptagonal, octogonal, etc. Justifique essa afirmação.

EXERCÍCIO 2

A definição do termo hexagonal é arbitrária, pois, dependendo da aplicação, ela poderia ser chamada de quadrangular, pentagonal, heptagonal, octogonal, etc. Justifique essa afirmação.

Em uma Arquitetura Hexagonal, cada face é um motivo para comunicação com o mundo externo. Logo, podemos ter 2, 3, 4,... faces

EXERCÍCIO 3

A seguir, mostramos duas classes de domínio, implementadas em Python usando Django. O código mostrado define regras para mapeamento dos campos de objetos dessas classes para linhas de tabelas de um BD relacional. Esta implementação segue os princípios de uma arquitetura hexagonal? Justifique.

```
1 class Musician(models.Model):
2     first_name = models.CharField(max_length=50)
3     last_name = models.CharField(max_length=50)
4     instrument = models.CharField(max_length=100)
5
6 class Album(models.Model):
7     artist = models.ForeignKey(Musician, on_delete=models.CASCADE)
8     name = models.CharField(max_length=100)
9     release_date = models.DateField()
10    num_stars = models.IntegerField()
```

Arquiteturas atuais

Domain-Driven Design (DDD)

Arquitetura Hexagonal

Arquitetura Limpa (Clean Architecture)

Injeção de Dependência

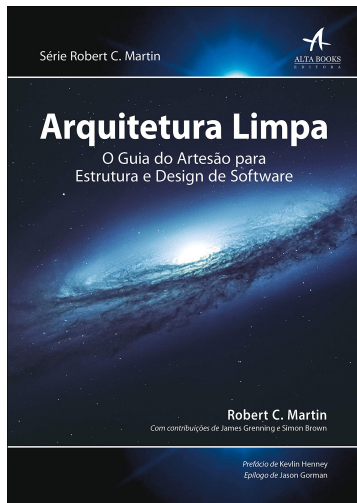
Padrão Composite

Arquitetura Serverless

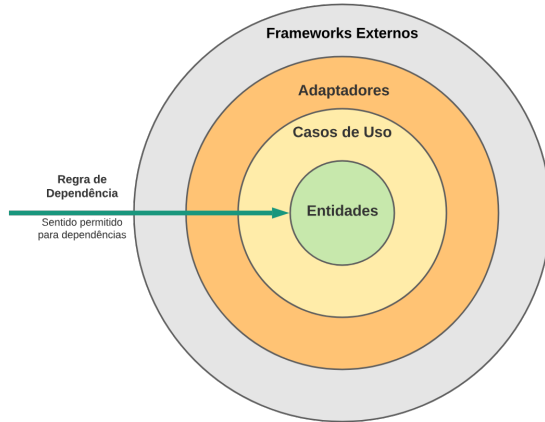
ARQUITETURA LIMPA (CLEAN ARCHITECTURE)

Padrão arquitetural proposto por Robert Martin **Mesmo objetivo de uma Arquitetura Hexagonal**

- ▶ Separação entre código de domínio e de tecnologia
- ▶ Código de domínio “limpo” de código de tecnologia



ARQUITETURA LIMPA



Domínio = entidades + casos de uso

ENTIDADES

Classes que representam objetos importantes do domínio

Têm vida própria e um identificador

Podem ser compartilhadas por mais de um sistema

Exemplos: Livro, Aluno, Professor, Disciplina, Departamento

CASOS DE USO

São classes que implementam os serviços que o sistema oferece; fazem todas as validações de regras de negócio

Exemplos:

- ▶ Criar disciplina
- ▶ Matricular aluno em disciplina
- ▶ Cancelar matrícula de aluno
- ▶ Lançar notas de disciplinas

OBSERVAÇÃO

- ▶ Casos de uso em Arq. Limpa não tem relação direta com casos de uso para especificação de requisitos
- ▶ Menos relação ainda com Diagramas de Casos de Uso

Transferir Valores entre Contas

Ator: Cliente do Banco

Fluxo normal:

1 - Autenticar Cliente

2 - Cliente informa agência e conta de destino da transferência

3 - Cliente informa valor que deseja transferir

4 - Cliente informa a data em que pretende realizar a operação

5 - Sistema efetua transferência

6 - Sistema pergunta se o cliente deseja realizar uma nova transferência

Extensões:

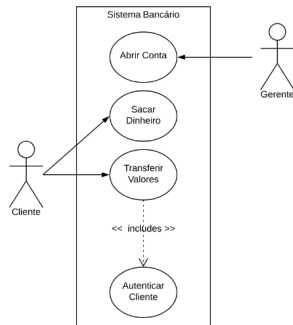
2a - Se conta e agência incorretas, solicitar nova conta e agência

3a - Se valor acima do saldo atual, solicitar novo valor

4a - Data informada deve ser a data atual ou no máximo um ano a frente

5a - Se data informada é a data atual, transferir imediatamente

5b - Se data informada é uma data futura, agendar transferência



ADAPTADORES

Mesmo objetivo dos adaptadores de uma Arq. Hexagonal

Mediar a interação entre:

- ▶ Sistemas externos
- ▶ Domínio: Casos de Uso e Entidades

MESMA COISA DA ARQUITETURA HEXAGONAL

- Frontend (Web, mobile, etc)
- Bancos de dados
- Sistemas externos (gateways pagto, etc)
- Bibliotecas de terceiros (logging, cache, pub/sub, etc)



Adaptadores

Domínio “limpo” de tecnologia

SISTEMAS E FRAMEWORKS EXTERNOS

BDs, interface com usuário, envio de mails, provedores de pagamento, comunicação com hardware, etc.



Todos os detalhes ficam na camada de frameworks e drivers. A Web é um detalhe. O banco de dados é um detalhe. Mantemos essas tecnologias na camada mais externa porque é onde elas podem fazer menos mal.

REGRA DE DEPENDÊNCIA

Um elemento declarado em uma camada externa não deve ser mencionado pelo código de uma camada interna.

“Camada interna não conhece sua camada externa”

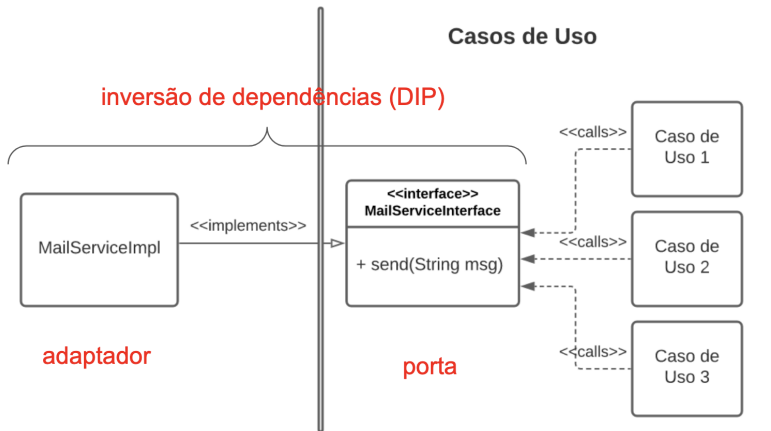
- ▶ Entidades não conhecem os casos de uso
- ▶ Casos de uso não conhecem os adaptadores

EXEMPLO DE REGRA DE DEPENDÊNCIA

Classe interna (caso de uso) tem que chamar um método de uma classe externa (envio de mail)

EXEMPLO DE REGRA DE DEPENDÊNCIA

Classe interna (caso de uso) tem que chamar um método de uma classe externa (envio de mail)



INTERFACE NO CASO DE USO

```
1 package casosDeUso;
2
3 public interface MailServiceInterface {
4     void send(String msg);
5 }
6
7 public class CasoDeUso1 {
8     private MailServiceInterface ms;
9
10    public CasoDeUso1(MailServiceInterface ms) {
11        this.ms = ms;
12    }
13
14    public void algumMetodoNegocio() {
15        ... this.ms.send("corpo do mail"); ...
16    }
17 }
```

IMPLEMENTAÇÃO NO ADAPTADOR

```
1 package adaptadores;  
2  
3 import casosDeUso.MailServiceInterface;  
4  
5 public class MailServiceImpl implements MailServiceInterface {  
6     public void send(String msg) {  
7         // envia e-mail usando sistema externo  
8     }  
9 }
```

CLASSE PRINCIPAL

```
1 package main;
2
3 import adaptadores.MailServiceImpl;
4 import casosDeUso.CasoDeUso1;
5
6 class ClassePrincipal {
7     public static void main() {
8         new CasoDeUso1(new MailServiceImpl());
9     }
10 }
```

EXERCÍCIO 1

Em uma arquitetura limpa o nome de um elemento declarado em uma camada externa não deve ser mencionado pelo código de uma camada interna. Qual a principal vantagem ou benefício dessa regra?

EXERCÍCIO 1

Em uma arquitetura limpa o nome de um elemento declarado em uma camada externa não deve ser mencionado pelo código de uma camada interna. Qual a principal vantagem ou benefício dessa regra?

Blindar as camadas internas (principalmente, Entidades e Casos de Uso) de mudanças na tecnologia.

EXERCÍCIO 2

Suponha que um sistema use tecnologias X, Y e Z. E suponha que temos certeza de que elas nunca vão mudar no futuro. Ou seja, não existe chance de amanhã o sistema ter que usar uma tecnologia X', Y' ou Z'. Nesse cenário, você acha que ainda pode ser útil a adoção de uma Arquitetura Limpa? Justifique.

EXERCÍCIO 2

Suponha que um sistema use tecnologias X, Y e Z. E suponha que temos certeza de que elas nunca vão mudar no futuro. Ou seja, não existe chance de amanhã o sistema ter que usar uma tecnologia X', Y' ou Z'. Nesse cenário, você acha que ainda pode ser útil a adoção de uma Arquitetura Limpa? Justifique.

SIM

- ▶ Testabilidade. Fica mais fácil testar as classes das camadas limpas de tecnologia
- ▶ Compreensão do código. O entendimento das camadas fica mais simples também.

EXERCÍCIO 3

Supondo um Sistema de Bibliotecas que usa uma Arquitetura Limpa, responda as perguntas abaixo:

- ▶ A classe Livro deve pertencer a qual camada?
- ▶ No sistema, existe uma interface de negócio com um método que retorna os livros emprestados para um certo usuário. Essa interface pertence a qual camada?
- ▶ Sobre a interface anterior, existe uma classe que implementa os seus métodos (e que usa SQL para acessar o BD). Essa classe está em qual camada?
- ▶ Para terminar, o SGBD estará localizado em qual camada?

Arquiteturas atuais

Domain-Driven Design (DDD)

Arquitetura Hexagonal

Arquitetura Limpa (Clean Architecture)

Injeção de Dependência

Padrão Composite

Arquitetura Serverless

OBJETIVO

Vamos dar uma olhada em dois padrões de projeto:

- ▶ Injeção de dependência
- ▶ Composite

INJEÇÃO DE DEPENDÊNCIAS

Padrão de projeto (porém não é um padrão GoF)

Usado para desacoplar uma classe de suas dependências mais importantes e sujeitas a mudanças

EXEMPLO

```
1 class A {  
2     private ServicoPagto pagto; // A depende de ServicoPagto  
3     ...  
4 }
```

EXEMPLO

```
1 class A {  
2     private ServicoPagto pagto; // A depende de ServicoPagto  
3     ...  
4 }
```

```
1 class A {  
2     private ServicoPagto pagto = new PagtoPayPal();  
3     ...  
4 }
```

Qual o problema?

EXEMPLO

```
1 class A {  
2     private ServicoPagto pagto; // A depende de ServicoPagto  
3     ...  
4 }
```

```
1 class A {  
2     private ServicoPagto pagto = new PagtoPayPal();  
3     ...  
4 }
```

Qual o problema? Dependência está “hard-coded”

SOLUÇÃO QUE USA INJEÇÃO DE DEPENDÊNCIA (VIA UM CONSTRUTOR)

```
1 class A {  
2     private ServicoPagto pagto;  
3  
4     A(ServicoPagto pagto) { // injeção de dependência  
5         this.pagto = pagto;  
6     }  
7     ...  
8 }
```

SOLUÇÃO QUE USA INJEÇÃO DE DEPENDÊNCIA (VIA UM MÉTODO SET)

```
1 class A {  
2     private ServicoPagto pagto;  
3     ...  
4  
5     void setServicoPagto(ServicoPagto pagto) {  
6         this.pagto = pagto;  
7     }  
8     ...  
9 }
```

INJEÇÃO DE DEPENDÊNCIA: RESUMO

Injetar dependências: parametrizar uma classe de forma que seja possível alterar suas dependências.

Objetivo: permitir mudar as dependências de uma classe

FRAMEWORKS DE INJEÇÃO DE DEPENDÊNCIA

Assumem a responsabilidade de:

- ▶ Criar uma classe A
- ▶ Criar e injetar as dependências de A

Exemplos:

- ▶ Java: Spring, Guice, Dagger2
- ▶ TypeScript: InversifyJS
- ▶ Go: wire

EXEMPLO: CLASSE NA QUAL A DEPENDÊNCIA SERÁ INJETADA

```
1 class A {  
2     private ServicoPagto pagto;  
3  
4     @Inject          // anotação do framework  
5     A(ServicoPagto pagto) {  
6         this.pagto = pagto;  
7     }  
8     ...  
9 }
```

EXEMPLO: CLASSE QUE CRIA A CLASSE DA DEPENDÊNCIA

```
1 class Cliente {  
2     void foo() {  
3         A a = DIF.getInstance(A.class); // DIF é um nome hipotético  
4         ...  
5     }  
6 }
```

COMO O FRAMEWORK VAI INFERIR A DEPENDÊNCIA QUE PRECISA INJETAR NA CLASSE A?

COMO O FRAMEWORK VAI INFERIR A DEPENDÊNCIA QUE PRECISA INJETAR NA CLASSE A?

Esta informação é declarada em um arquivo externo

Exemplo (hipotético):

```
1 { "dependencies" : {  
2   "ServicoPagto": {  
3     "class": "PgtoPayPal"  
4   }  
5   ...  
6 }}
```


PERGUNTA: QUAL A PRINCIPAL VANTAGEM DE USAR UM DIF?



PERGUNTA: QUAL A PRINCIPAL VANTAGEM DE USAR UM DIF?

Reduz o acoplamento e aumenta a testabilidade

- ▶ torna o sistema mais flexível e fácil de manter

JAVA/SPRING

```
1 public interface MessageSender {
2     void send(String message);
3 }
4 @Component
5 @Primary
6 public class EmailSender implements MessageSender {
7     @Override
8     public void send(String message) {
9         System.out.println("Enviando e-mail: " + message);
10    }
11 }
12 @Component
13 public class SmsSender implements MessageSender {
14     @Override
15     public void send(String message) {
16         System.out.println("Enviando SMS: " + message);
17    }
18 }
```

JAVA/SPRING

```
1 @Service
2 public class NotificationService {
3
4     private final MessageSender sender;
5
6     // O Spring injeta a dependência automaticamente
7     @Autowired
8     public NotificationService(MessageSender sender) {
9         this.sender = sender;
10    }
11
12    public void send(String message) {
13        sender.send(message);
14    }
15 }
```

Arquiteturas atuais

Domain-Driven Design (DDD)

Arquitetura Hexagonal

Arquitetura Limpa (Clean Architecture)

Injeção de Dependência

Padrão Composite

Arquitetura Serverless

PADRÃO COMPOSITE

- ▶ Padrão de projeto GoF
- ▶ Simples e muito usado
- ▶ Permite:
 - Agrupar objetos
 - Tratar agrupamento como objeto “normal”

Design Patterns

Elements of Reusable
Object-Oriented Software

Erich Gamma
Richard Helm
Ralph Johnson
John Vlissides



Cover art © 1994 M.C. Escher / Cordon Art - Baarn - Holland. All rights reserved.

Foreword by Grady Booch

ADDISON-WESLEY PROFESSIONAL COMPUTING SERIES



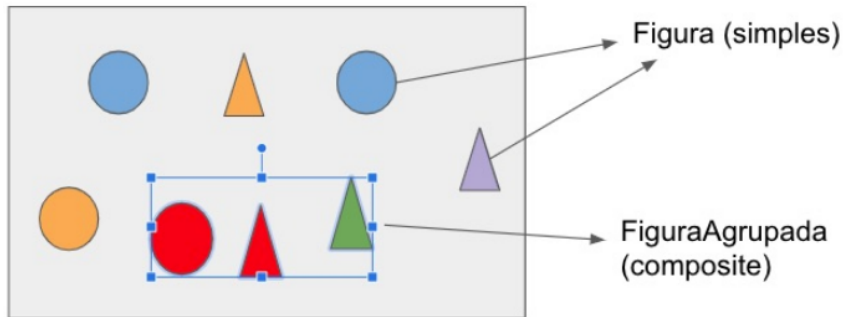
CONTEXTO

Editor gráfico que permite desenhar Circulo, Triangulo, etc.

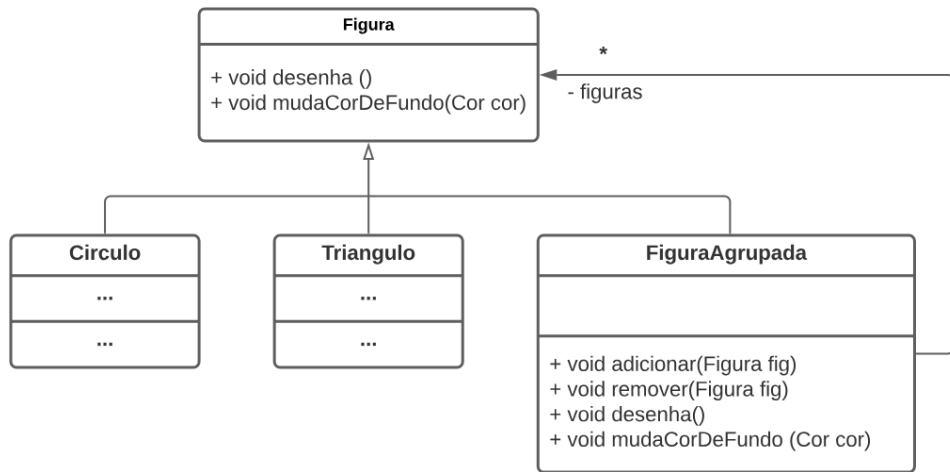
```
1 interface Figura {  
2     void desenha();  
3     void mudaCorDeFundo(Cor cor);  
4 }  
5  
6 class Circulo implements Figura {  
7     ...  
8 }  
9  
10 class Triangulo implements Figura {  
11     ...  
12 }
```

PROBLEMA

Ficamos encarregados de implementar uma funcionalidade que permita agrupar figuras e tratar a figura resultante como uma figura “normal”



SOLUÇÃO: PADRÃO COMPOSITE



SOLUÇÃO: PADRÃO COMPOSITE

Interface Comum

```
1 interface Figura {  
2     void desenha();  
3     void mudaCorDeFundo(Cor cor);  
4 }
```

SOLUÇÃO: PADRÃO COMPOSITE

```
1 class FiguraAgrupada implements Figura {
2     private ArrayList<Figura> figuras = new ArrayList<Figura>();
3     public void adicionar(Figura fig) {
4         figuras.add(fig);
5     }
6     public void remover(Figura fig) {
7         figuras.remove(fig);
8     }
9     public void desenha() {
10         for (Figura fig: figuras) {
11             fig.desenha();
12         }
13     }
14     public void mudaCorDeFundo(Cor cor) {
15         for (Figura fig: figuras) {
16             fig.mudaCorDeFundo(cor);
17         }
18     }
19 }
```

CÓDIGO QUE USA O PADRÃO COMPOSITE

```
1 class Cliente {  
2     void foo(Figura fig) {  
3         ...  
4         fig.desenha(); // desenha figura simples ou agrupada  
5         ...  
6     }  
7 }
```

CÓDIGO QUE USA O PADRÃO COMPOSITE

```
1 class Main {
2     public main() {
3         Cliente cliente = new Cliente();
4         Circulo c1 = new Circulo();
5         cliente.foo(c1); // chama foo com figura simples
6
7         Triangulo t1 = new Triangulo();
8         Triangulo t2 = new Triangulo();
9         FiguraAgrupada grupo = new FiguraAgrupada();
10        grupo.adicionar(c1);
11        grupo.adicionar(t1);
12        grupo.adicionar(t2);
13        cliente.foo(grupo); // chama foo com figura agrupada
14    }
15 }
```

EXERCÍCIO 1

Injeção de Dependência, muitas vezes, é comparada com padrão Fábrica. Qual a desvantagem de configurar dependências por meio de fábricas? Para responder, compare os seguintes códigos:

```
1 class A {  
2     IB b;  
3  
4     A(IB b) {  
5         this.b = b;  
6     }  
7 }
```

```
1 class A {  
2     IB b;  
3  
4     A(IB b) {  
5         this.b = IB_Factory.getInstance();  
6     }  
7 }
```

EXERCÍCIO 1

Injeção de Dependência, muitas vezes, é comparada com padrão Fábrica. Qual a desvantagem de configurar dependências por meio de fábricas? Para responder, compare os seguintes códigos:

```
1 class A {  
2     IB b;  
3  
4     A(IB b) {  
5         this.b = b;  
6     }  
7 }
```

- Pode receber objetos dos tipos B1, B2, etc.

```
1 class A {  
2     IB b;  
3  
4     A(IB b) {  
5         this.b = IB_Factory.getInstance();  
6     }  
7 }
```

- Sempre cria objetos de um tipo B

EXERCÍCIO 2

Por que costuma-se dizer que Injeção de Dependência pode, em certos casos, violar Ocultamento de Informação (encapsulamento)?

EXERCÍCIO 2

Por que costuma-se dizer que Injeção de Dependência pode, em certos casos, violar Ocultamento de Informação (encapsulamento)?

```
1 public class OrderService {  
2     private final PaymentGateway gateway;  
3     private final DiscountPolicy policy;  
4  
5     public OrderService(PaymentGateway gateway, DiscountPolicy policy) {  
6         this.gateway = gateway;  
7         this.policy = policy;  
8     }  
9 }
```

- Qualquer um que queira usar OrderService já fica sabendo que ele depende de um gateway de pagamento e de uma política de desconto.

EXERCÍCIO 3

Existem 3 tipos de classes/interfaces no padrão Composite:

- ▶ Uma interface visível para o cliente (Figura)
- ▶ Classes de objetos simples (Circulo e Triangulo)
- ▶ Classe responsável pela composição (FiguraAgrupada)

Considerando essas classes, pense em um outro exemplo de Composite e responda:

- ▶ Qual é a interface visível para o cliente? Quais métodos ela define?
- ▶ Quais são as classes de objetos simples? Basta citar o nome delas.
- ▶ Qual a classe responsável pela composição? Basta citar o nome dela.

Arquiteturas atuais

Domain-Driven Design (DDD)

Arquitetura Hexagonal

Arquitetura Limpa (Clean Architecture)

Injeção de Dependência

Padrão Composite

Arquitetura Serverless

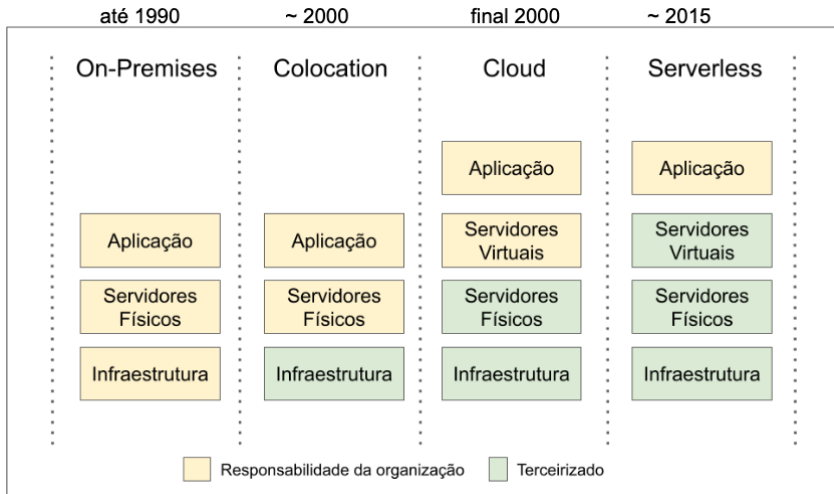
DDD, Arquitetura Hexagonal, Arquitetura Limpa, etc

- ▶ Preocupação: organização modular de um sistema

Serverless:

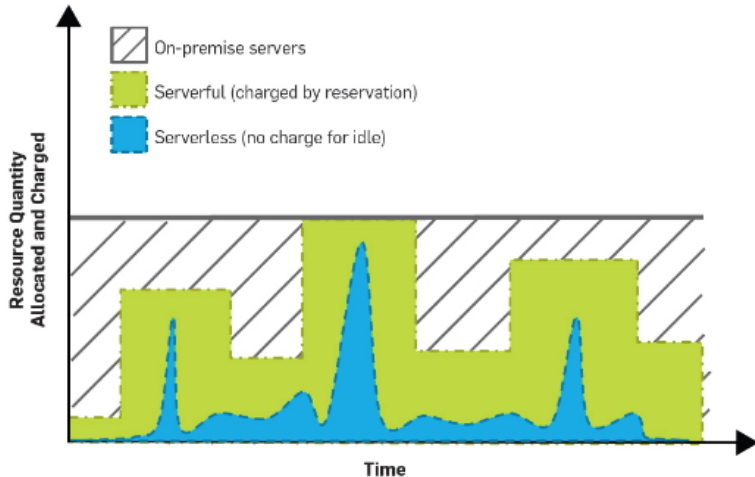
- ▶ Visão tecnológica e voltada para plataformas de cloud
- ▶ Uma evolução de plataformas de cloud

HISTÓRICO (DATAS APROXIMADAS)



MODELO DE PAGAMENTO

Por tempo de execução (inspirado em serviços de água ou energia elétrica)



FUNÇÕES SERVERLESS

Invocadas explicitamente ou quando ocorrer um evento (exemplo: novo registro adicionado no BD)

Stateless (mas podem acessar BDs, filas de msgs, etc)

Possuem um tempo de execução máximo (ex.: 15 min), após esse tempo são automaticamente canceladas

Podem ser implementadas em várias linguagens

Nomes específicos da plataforma de cloud (exemplo: funções lambda, no caso da AWS)

EXEMPLO (NETLIFY)

```
1 exports.handler = async (event, context) => {  
2     return {  
3         statusCode: 200,  
4         body: "Hello, World"  
5     };  
6 };
```

[<https://functions.netlify.com/.netlify/functions/hello>]

ARQUITETURA SERVERLESS

Pequenas funções, que executam rapidamente

Modelo de comunicação assíncrono:

- ▶ Função processa, produz alguns eventos e termina
- ▶ Esses eventos disparam a execução de outras funções

VANTAGENS DE SERVERLESS

Custo pode ser menor

Escalabilidade

Não precisamos nos preocupar com configuração de máquinas virtuais e configuração de parâmetros do cloud

DESVANTAGENS DE SERVERLESS

Complexidade arquitetural

Maior latência, principalmente na primeira execução (cold start problem)

Dependência de fornecedores (vendor lock-in).

EXERCÍCIO 1

Suponha uma agenda de compromissos construída usando-se serverless. Mostra-se ao lado uma das funções dessa aplicação, a qual retorna todos os compromissos inseridos na agenda. Qual a desvantagem de serverless fica mais clara ao analisarmos o código dessa função? Justifique.

```
1 'use strict';
2
3 const AWS = require('aws-sdk');
4 const dynamoDb = new AWS.DynamoDB.DocumentClient();
5
6 module.exports = (event, callback) => {
7   const params = {
8     TableName: 'todos',
9   };
10
11   return dynamoDb.scan(params, (error, data) => {
12     if (error) {
13       callback(error);
14     }
15     callback(error, data.Items);
16   });
17 };
```

EXERCÍCIO 2

Analise os diagramas de sequência mostrados a seguir. Eles são de uma aplicação que inicialmente estava implementada usando uma arquitetura monolítica (figura 1) e que foi migrada para usar funções serverless (figura 2). A funcionalidade mostrada permite que um usuário faça o upload de um arquivo e solicite a sua conversão para um outro formato (pdf, svg, etc). Responda:

- ▶ Qual dos dois diagramas é mais simples e fácil de entender? Justifique sua resposta.
- ▶ Descreva uma diferença importante entre os dois diagramas. Ou seja, descreva uma mudança relevante que teve que ser realizada na estrutura original da aplicação (figura 1) para torná-la adequada para uma arquitetura baseada em serverless (figura 2). Justifique sua resposta.

EXERCÍCIO 2

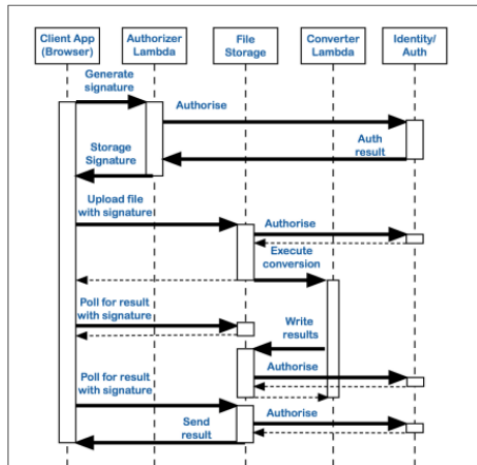
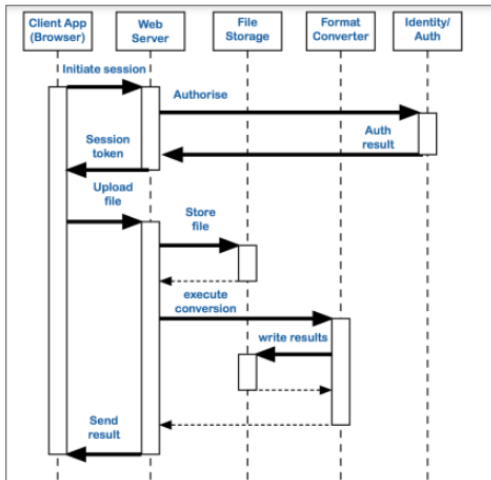


Figure 2: MindMup file conversion with Lambda

Este documento está licenciado sob a Licença Atribuição-NãoComercial-Compartilhalgual 4.0 Internacional Creative Commons. Para visualizar uma cópia desta licença, visite <http://creativecommons.org/licenses/by-nc-sa/4.0/> ou mande uma carta para Creative Commons, PO Box 1866, Mountain View, CA 94042, USA.

